

长期记忆对话 Agent 中期进展报告

汤岑宸 学号：231880523 曹信阳 学号：231880114

2026 年 6 月

1 项目目标

本项目旨在实现一个支持长期记忆的对话 Agent，使其能够在跨会话、跨时间的交互中保存并利用历史信息。系统需要从多轮、多会话对话中抽取结构化记忆，在回答新问题时检索相关记忆，并通过记忆更新、冲突处理或遗忘机制提升长期对话问答能力。

与普通上下文增强方法不同，本项目重点不只是把原始对话直接拼入 prompt，而是将原始对话日志加工为可管理、可追踪、可检索的记忆单元。最终系统将在基于 LoCoMo 的长期对话评测集上进行评估，并与 No-memory、Full-context 和 Vanilla RAG 等基线方法进行对比。

2 当前进度

截至中期提交阶段，项目已完成以下工作：

1. 完成任务需求分析，明确长期记忆 Agent 的核心目标，包括记忆写入、记忆存储、记忆检索、记忆更新和回答生成。
2. 阅读并整理课程提供的项目说明与评测工具包，明确统一实验流程：准备评测集、运行 Agent 生成预测、使用 LLM-as-Judge 评测结果。
3. 完成 Python 虚拟环境配置，安装 PyTorch、sentence-transformers、OpenAI SDK 和 vLLM。当前机器可识别 NVIDIA GeForce RTX 4060 Laptop GPU，并可运行 CUDA 推理。
4. 使用本地下载的 Qwen2.5-3B-Instruct-AWQ 通过 vLLM 提供 OpenAI 兼容接口作为生成模型。Vanilla RAG 使用本地 BAAI/bge-m3 embedding 模型进行向量检索，embedding 计算设置为 CPU，LLM 推理使用 GPU。
5. 使用助教脚本 `prepare_eval_set.py` 生成 200 问题评测集，并完成 No-memory、Full-context、Vanilla RAG 三组 baseline 以及当前自定义 MemoryAgent 的预测生成。
6. 使用 DeepSeek 线上 API 和助教脚本 `run_judge.py` 完成 LLM-as-Judge 正式测评，获得当前阶段的主指标、调用成本和端到端响应时间。

需要说明的是，当前工具包本地缓存数据源为 `locomomo10.json`，因此本阶段生成的评测集包含 10 段可用对话、200 个问题，并保证四类问题各 50 个。

3 系统方案设计

当前实现的长期记忆 Agent 采用“写入-更新-存储-检索-生成”的流水线结构。整体目标不是直接把全部原始对话塞入 prompt，而是在 **ingest** 阶段先把历史对话转化为可管理的长期记忆单元，再在 **answer** 阶段检索相关记忆辅助回答。

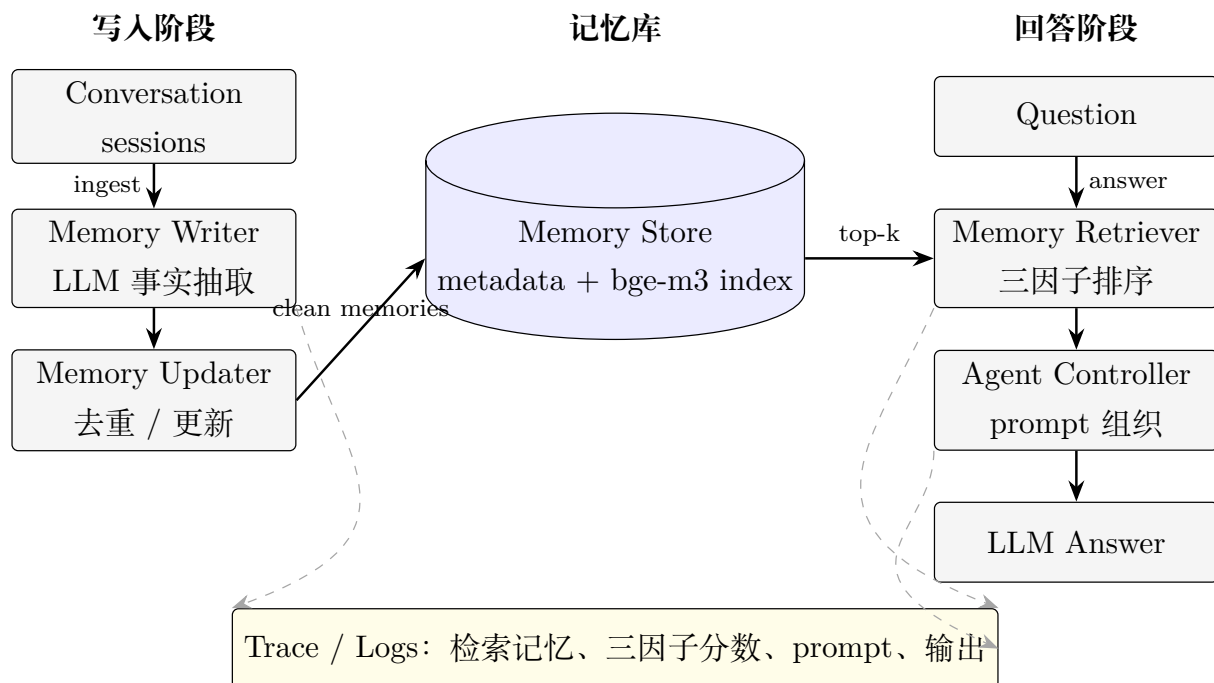


图 1: 自定义 MemoryAgent 的整体架构

其中，Memory Writer 和 Memory Updater 主要参考 Mem0 的“记忆提取-冲突检测-更新”流程；Memory Retriever 参考 Generative Agents 的 relevance、recency 和 importance 三因子检索；Memory Store 中的时间衰减是对 MemoryBank 遗忘思想的轻量近似。当前版本优先保证结构完整、日志可追踪和实验可消融，而不是追求复杂度。

表 1: 自定义 MemoryAgent 架构

模块	输入 / 输出	当前实现
Memory Writer	输入原始 session，输出结构化记忆	使用本地 Qwen 对每个 session 分块抽取 JSON 记忆，字段包括 text、subject、attribute、importance、source、timestamp
Memory Updater	输入候选记忆，输出清洗后的记忆	文本级去重；对相同 subject + attribute 的记忆保留较新的版本
Memory Store	输入记忆单元，输出可检索索引	使用列表保存元数据，用 BAAI/bge-m3 编码记忆文本，矩阵乘法计算相似度

模块	输入 / 输出	当前实现
Memory Retriever	输入 query, 输出 top-k 记忆	使用 relevance、recency、importance 三因子加权排序, 默认权重为 0.70、0.15、0.15
Agent Controller	输入问题, 输出最终答案	串联写入、更新、存储、检索和生成; 记录检索结果、prompt、三因子分数和 LLM 输出

3.1 Memory Writer

Memory Writer 负责从原始多会话对话中抽取结构化记忆。当前版本参考 Mem0 的工程流程, 使用本地 Qwen2.5-3B-Instruct-AWQ 对每个 session 分块抽取 JSON 形式的记忆单元。每条记忆包含:

- **text**: 一条独立、简短的事实性记忆;
- **subject**: 记忆涉及的人或实体;
- **attribute**: 记忆类型, 如 preference、location、job、event、relationship、plan、update 等;
- **importance**: 1-5 的重要性分数;
- **source** 和 **timestamp**: 用于追踪来源和时间。

该模块避免直接把原始对话切片作为最终记忆, 而是尝试将对话压缩为更清晰的事实性记忆。这样做的优点是记忆更短、更容易管理; 缺点是如果 Writer 漏掉了关键事实, 后续检索和生成都无法恢复该信息。

3.2 Memory Store

Memory Store 负责保存结构化记忆, 并为检索提供索引。当前版本使用列表结构保存元数据, 同时使用本地 BAAI/bge-m3 生成向量表示, 并通过矩阵乘法计算 query 与记忆之间的余弦相似度。每条记忆还保存插入顺序、访问次数、时间戳和重要性等字段, 方便后续加入遗忘曲线、访问增强或 FAISS / Chroma 索引。

3.3 Memory Retriever

Memory Retriever 在回答问题时根据 query 检索相关记忆。当前版本参考 Generative Agents, 引入 relevance、recency 和 importance 三因子排序:

$$\text{Score}(m, q) = 0.70 \cdot \text{Relevance}(m, q) + 0.15 \cdot \text{Recency}(m) + 0.15 \cdot \text{Importance}(m)$$

其中, Relevance 来自 bge-m3 embedding 相似度; Recency 使用时间衰减近似, 越新的记忆分数越高; Importance 来自 Writer 抽取阶段给出的 1-5 分并归一化。当前默认返回 top-6 条记忆, 并在 trace 中保存每条记忆的总分和三个子分数, 方便后续分析检索错误。

3.4 Memory Updater

记忆管理机制当前实现了轻量去重和更新。当两条记忆文本规范化后相同时，只保留一条并保留较高 importance；当两条记忆具有相同的 `subject + attribute` 时，系统保留时间更新的记忆。该机制对应 Mem0 中的冲突检测与更新思想，但当前没有额外调用 LLM 判别冲突，因此成本较低。

3.5 Agent Controller

Agent Controller 负责组织完整流程。对于每段对话，评测脚本先调用 `ingest(conversation)`，此时系统完成记忆写入、更新和索引构建；随后对该对话下的每个问题调用 `answer(question)`，Controller 检索相关记忆并构造回答 prompt。

当前回答 prompt 只包含检索出的结构化记忆，而不直接包含原始对话片段。这样可以测试“结构化长期记忆”本身是否有效，但也带来了当前 Ours 效果较差的核心风险：一旦 Writer 抽取时丢失细节，Generator 无法从原始对话中补救。因此，下一步计划加入“结构化记忆 + 原始证据片段”的回退机制。

4 初步实验设置

本阶段实验使用助教提供的统一脚本和接口规范。生成阶段使用本地 vLLM 部署的 Qwen2.5-3B-Instruct-AWQ，Judge 阶段使用 DeepSeek 线上 API 的 `deepseek-v4-flash`。Vanilla RAG 的检索模型为本地 BAAI/bge-m3。所有 baseline 使用同一底座生成模型，变化只发生在记忆使用方式上。

表 2: 当前方法设置

方法	记忆使用方式	说明
No-memory	不使用历史对话	仅将当前问题输入 LLM
Full-context	截断后的历史上下文	将对话历史拼入 prompt
Vanilla RAG	原始对话切片检索	使用向量检索取 top-k 片段辅助回答
Ours	结构化记忆检索	LLM 写入记忆，三因子检索，轻量更新

评测指标包括三类：第一，QA 准确率，即助教脚本基于 LLM-as-Judge 计算的 Judge Score，这是本项目主指标；第二，平均每题 LLM 调用次数，用于衡量系统调用成本；第三，平均端到端响应时间，用于衡量系统效率。

5 正式测评结果

表 3 给出了当前 200 问题评测集上的正式 LLM-as-Judge 结果。Judge 输出标签包括 CORRECT、PARTIAL 和 WRONG，对应分数分别为 1.0、0.5 和 0.0，总体得分为所有题目得分的平均值。

表 3: 当前方法的正式 Judge 结果

方法	Judge Score	F1	EM	平均 LLM 调用	平均端到端时间
No-memory	0.0000	0.0090	0.000	1.0	0.064s
Full-context	0.0500	0.0547	0.005	1.0	0.865s
Vanilla RAG	0.1300	0.1455	0.015	1.0	3.269s
Ours	0.0275	0.0653	0.010	2.69	12.785s

从总体结果看, Vanilla RAG 的 Judge Score 最高, 为 0.1300; Full-context 为 0.0500; No-memory 为 0.0000; 当前自定义 MemoryAgent 仅为 0.0275。该结果说明, 长期记忆任务中历史信息非常关键, 而直接检索原始对话片段的 Vanilla RAG 在当前阶段反而最稳。我们的系统虽然结构更完整, 但因为 Memory Writer 会先压缩原始对话, 若抽取阶段丢失关键信息, 后续三因子检索无法恢复这些证据。同时, Ours 的平均每题 LLM 调用次数达到 2.69, 端到端时间为 12.785s, 说明当前结构化记忆写入带来了明显成本, 但尚未带来准确率收益。

表 4: 按问题类型拆分的 Judge Score

方法	multi-hop	open-domain	single-hop	temporal
No-memory	0.000	0.000	0.000	0.000
Full-context	0.000	0.180	0.020	0.000
Vanilla RAG	0.040	0.270	0.180	0.030
Ours	0.000	0.060	0.050	0.000

分类结果显示, Vanilla RAG 在 open-domain 和 single-hop 问题上提升最明显, 分别达到 0.270 和 0.180; 但所有方法在 multi-hop 和 temporal 上仍然较弱。这与长期记忆任务的难点一致: 时间推理需要显式建模事件发生时间、更新时间和先后关系; 多跳问题则要求系统跨多个会话片段整合信息。Ours 在 temporal 和 multi-hop 上均为 0, 说明当前结构化记忆抽取没有稳定保留多跳证据和时间线信息。

6 当前 MemoryAgent 效果较差的原因分析

当前自定义 MemoryAgent 的效果明显低于 Vanilla RAG, 主要原因并不是检索模型 `bge-m3` 本身, 而是结构化记忆流水线中存在信息瓶颈。

1. **写入阶段信息丢失。** Vanilla RAG 直接保留原始对话片段, 而 Ours 先让本地 3B 模型把 session 压缩为少量记忆句子。长期记忆问答中的答案往往依赖细节, 例如具体日期、事件对象、计划变化或一句短回复; 这些细节在抽取时容易被概括掉。一旦 Writer 没有写入某个事实, Retriever 和 Generator 后面都无法恢复。

2. **本地小模型不擅长稳定 JSON 记忆抽取。** Qwen2.5-3B-Instruct-AWQ 适合作为轻量生成模型，但用于长 session 的结构化事实抽取时，容易出现过度总结、遗漏实体、attribute 标注粗糙等问题。这会导致 Store 中的记忆看起来合理，但不一定覆盖评测问题所需答案。
3. **三因子检索的 recency 和 importance 目前偏粗。** 当前所有高层记忆往往 importance 都较高，importance 区分度不足；时间戳解析和 recency 衰减也只是近似实现。这样三因子排序并没有充分发挥 Generative Agents 中的优势，反而可能把较新的泛化记忆排在真正相关的旧事实前面。
4. **成本高但有效证据少。** Ours 在 ingest 阶段需要额外调用 LLM 抽取记忆，因此平均每题 LLM 调用次数达到 2.69，端到端时间也显著增加。但这些额外调用主要用于压缩历史，而不是直接回答问题；如果压缩质量不足，就会形成“更贵但更少证据”的系统。

因此，当前结果说明：系统架构已经跑通，但 Memory Writer 还不是一个可靠的信息保留模块。后续改进应优先降低写入损失，而不是继续增加复杂检索权重。

7 结果水平与阶段性判断

结合当前任务和 LoCoMo 风格长期记忆问答的难度，可以将 Judge Score 粗略理解为以下几个区间：

表 5: Judge Score 的经验性解释

Judge Score 区间	效果判断
0.00–0.05	基本未解决长期记忆问题，接近无记忆或随机回答
0.05–0.12	baseline 水平，能够答中少量简单题
0.12–0.25	有一定长期记忆能力，检索或记忆模块开始有效
0.25–0.40	效果较好，能够作为明显改进结果
0.40 以上	效果较强，通常需要高质量记忆写入、检索排序和更新机制

据此判断，当前 Vanilla RAG 的 0.1300 已经达到“有一定长期记忆能力”的初步区间，而 Ours 的 0.0275 仍接近无效系统。中期阶段该结果的意义主要在于：实验环境、数据准备、生成脚本、线上 Judge 流程和自定义 MemoryAgent 架构都已经完整跑通；同时结果也暴露出，结构化记忆并不会天然优于原始 RAG，关键在于 Writer 是否能高保真保留答案证据。后续自定义 MemoryAgent 应首先追平并超过 Vanilla RAG (0.1300)，再进一步做三因子检索和更新机制的消融。

8 下一步计划

后续将围绕当前 MemoryAgent 的失败原因进行改进，重点包括：

1. 改进 Memory Writer，不再只输出高度压缩的摘要记忆，而是保留更多原子事实，尤其是人物、时间、地点、计划和偏好变化。

2. 设计“原始证据回退”机制：检索结构化记忆的同时保留对应原始对话片段，回答时同时提供 memory 和 source snippets，减少写入阶段造成的信息丢失。
3. 做检索策略消融，对比只使用 relevance、relevance + recency、relevance + recency + importance，判断三因子权重是否真的有效。
4. 改进时间处理，对 temporal 问题显式保留事件时间和更新顺序，而不是仅依赖 embedding 相似度。
5. 基于 trace 挑选 3-5 个 bad case，分别判断错误来自写入丢失、检索未命中还是生成未利用。

9 风险与解决方案

当前主要风险包括三类。第一，长期对话数据较长，Full-context 容易触发上下文长度限制或显存压力。解决方案是保留 vLLM 8K 上下文配置，并在 baseline 中使用固定截断策略。第二，LLM-as-Judge 存在 API 成本和速率限制。解决方案是先用小样本调试，正式评测时使用 DeepSeek API 并控制并发数。第三，结构化记忆抽取质量可能不稳定。解决方案是在小样本上人工检查记忆单元，并通过 bad case 将错误归因到写入失败、检索失败或生成失败。

10 总结

中期阶段已完成长期记忆 Agent 的需求分析、系统方案设计、环境配置、baseline 生成、自定义 MemoryAgent 实现和线上 LLM-as-Judge 测评。实验结果表明，Vanilla RAG 当前明显优于 No-memory、Full-context 和 Ours，说明在当前数据和本地 3B 生成模型设置下，直接保留原始对话片段比低质量结构化抽取更可靠。我们的 MemoryAgent 效果较差，主要原因是写入阶段压缩导致关键信息丢失，同时带来了额外 LLM 调用和延迟。下一阶段将重点改进 Memory Writer 的高保真事实保留能力，并加入原始证据回退机制，再通过消融实验验证三因子检索和更新机制是否真正带来增益。